

MediFlow - Estándares de Repositorio y Flujo de Contribución

v1.0

1. Estructura del repositorio

MediFlow utiliza una arquitectura **monorepo**, con separación lógica por componente:

```
/backend
  Spring Boot 3
  Java 21

/frontend
  React
  TypeScript
  Vite

/ai-core
  Python
  FastAPI
  Pydantic
  LangGraph o pipeline Python puro

/docs
  Arquitectura
  Contratos API
  Diagramas
  Documentación técnica

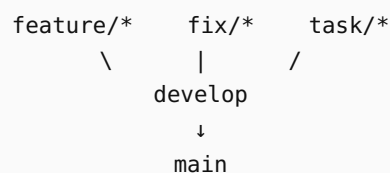
/test-data
  Documentos y datos sintéticos para pruebas del MVP
```

Los archivos de `/test-data` deberán utilizar únicamente **datos sintéticos**. No se incluirá información clínica real ni información personal de terceros.

La elección entre LangGraph y un pipeline Python puro corresponde al Squad IA Core, manteniendo los contratos definidos en `docs/ARCHITECTURE.md`.

2. Estrategia de ramas

El repositorio utiliza una estrategia de ramas simplificada. Las ramas de trabajo nacen desde `develop`, regresan mediante Pull Request y las versiones estables se promueven posteriormente a `main`.



2.1. `main`

Rama estable para demostraciones, entregas e hitos. No se realizan cambios de desarrollo directamente sobre ella. Recibe cambios desde `develop` mediante Pull Request.

2.2. `develop`

Rama principal de integración. Las ramas de trabajo se crean a partir de `develop` y regresan a ella mediante Pull Request.

2.3. Tipos de ramas de trabajo

Tipo	Uso	Ejemplo
feature/*	Nuevas funcionalidades del producto.	feature/12-document-upload
fix/*	Corrección de defectos o comportamientos incorrectos.	fix/18-file-validation
task/*	Configuración, infraestructura, documentación, contratos, datos de prueba y otras tareas técnicas.	task/1-init-backend

Los cambios puramente documentales utilizan normalmente `task/*`. Si la documentación forma parte de la corrección de un defecto del sistema, puede utilizarse `fix/*`.

3. Nomenclatura de ramas

Formato general:

```
feature/<issue>-<descripcion-corta>
fix/<issue>-<descripcion-corta>
task/<issue>-<descripcion-corta>
```

Los nombres deberán:

- incluir el número del issue correspondiente;
- usar letras minúsculas y `kebab-case`;
- evitar espacios, tildes y caracteres especiales;
- mantener una descripción corta y clara.

No se utilizarán identificadores manuales adicionales como `MF-01`; se utilizará directamente el número generado por GitHub.

4. Convención de commits

Se utilizará una versión simplificada de **Conventional Commits**:

Prefijo	Uso
feat:	Nueva funcionalidad.
fix:	Corrección de un defecto.
docs:	Cambios únicamente de documentación.
test:	Creación o modificación de pruebas.
refactor:	Modificación interna sin cambiar comportamiento.
chore:	Configuración, dependencias, Docker, tooling o mantenimiento.

```
feat: implement document upload endpoint
fix: validate unsupported file types
docs: update branching strategy
test: add urgent document scenario
refactor: simplify document validation flow
chore: configure backend dependencies
```

Los scopes como `feat(backend):` o `fix(frontend):` son opcionales.

4.1. Reglas para commits

- Cada commit debe representar un cambio coherente y utilizar un mensaje descriptivo.
- Evitar mensajes como `cambios`, `update`, `arreglo`, `final`, `ahora si` o `wip`.
- No incluir secretos, tokens, credenciales, claves API ni información sensible.
- Evitar mezclar trabajo de issues diferentes dentro del mismo commit cuando sea posible.
- No se exige un único commit por issue; pueden existir varios commits durante el desarrollo.

5. Pull Requests

Todo cambio destinado a `develop` o `main` deberá ingresar mediante Pull Request.

5.1. Título

```
[#<issue>] <descripción>

[#1] Inicializar servicio backend con Spring Boot
[#12] Implementar carga de documentos
```

5.2. Cuerpo

Como mínimo:

```
Issue: #<número>

## Qué cambia
Descripción breve del trabajo realizado.

## Cómo se validó
Pruebas o verificaciones realizadas.
```

El PR deberá vincularse manualmente con su issue mediante la sección **Development** de GitHub.

No se dependerá de expresiones como `Closes #1` para PR hacia `develop`, ya que `main` es la rama por defecto del repositorio.

6. Integración hacia `develop`

- Pull Request obligatorio.
- Al menos **1 aprobación**.
- Si se agregan nuevos commits, las aprobaciones anteriores dejan de ser válidas.
- Se bloquean la eliminación de la rama y los force-push.

- Se permiten **Merge** y **Squash**.
- Rebase no está permitido.
- No se exige Code Owners ni resolución técnica obligatoria de todos los comentarios.

Los comentarios que señalen errores, problemas de integración o cambios en contratos compartidos deberán atenderse antes del merge. Las sugerencias menores pueden quedar como mejoras posteriores si el equipo lo acuerda.

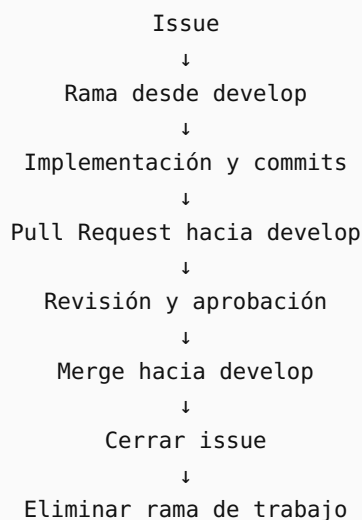
7. Integración hacia `main`

- Pull Request obligatorio.
- Al menos **1 aprobación**.
- Si se agregan nuevos commits, las aprobaciones anteriores dejan de ser válidas.
- Se bloquean la eliminación de la rama y los force-push.
- El único método permitido es **Merge**.
- Squash y Rebase no están permitidos.

Los cambios de `develop` hacia `main` se realizan **por hitos**, cuando exista una versión suficientemente estable para demostración o entrega. El merge final deberá ser coordinado por un Tech Lead.

El PR `develop` → `main` no deberá abrirse prematuramente mientras sigan entrando cambios relevantes de los squads para el mismo hito, salvo acuerdo del equipo.

8. Ciclo de vida de una tarea



Una tarea se considera terminada cuando su implementación está correctamente integrada y validada en `develop`. No es necesario esperar a que llegue a `main`, ya que esa rama se actualiza por hitos.

9. Cambios que afecten contratos compartidos

Los cambios que modifiquen endpoints, DTO, estructuras de datos, estados, enums, contratos Backend ↔ IA, contratos Backend ↔ Frontend u otras interfaces compartidas deberán comunicarse y acordarse previamente en el hilo:

No será suficiente comunicar un cambio únicamente por mensaje privado a otro integrante.

10. Convención de idioma

- Documentación, issues y Pull Requests: **español**.
- Nombres de ramas y mensajes de commit: **inglés**.
- Identificadores de código, endpoints, enums y variables: conservar el idioma y nomenclatura definidos en la arquitectura y los contratos técnicos.